SRI LANKA INSTITUTE OF INFORMATION TECHNOLOGY

Faculty of Computing

BSc (Hons) in Information Technology — Year 4, Semester 1



SE4041 — Mobile Application Design and Development

Assignment 02 — Final Report

Part A: KolaCare — an AI leaf doctor for Sri Lankan farmers (iOS)
Part B: KolaCare Field Lab — spatial crop-disease training (visionOS)

Item	Details
Student name	G M H Himal
Student ID	IT23152632
Module	SE4041 — Mobile Application Design and Development
Submission	October 2026

Executive summary

Crop pests and diseases destroy up to 40 % of the world's crops each year [1], and in Sri Lanka roughly a quarter of all workers earn their living from agriculture [2]. Smallholders usually notice disease late, cannot reach an Agriculture Instructor quickly, and cannot judge how severe an outbreak is or whether a treatment is working. This project answers that with two applications.

KolaCare (Part A, iOS 17+) turns a phone into a field diagnostic tool. A leaf photo passes through an on-device pipeline — Vision subject lifting, a pixel-level symptom analyser, and a Create ML image classifier run through Vision/CoreML — that names the disease, measures the percentage of the leaf that is damaged and draws a lesion map. Diagnoses are stored per plant with SwiftData, tracked over time with Swift Charts, mapped with MapKit and linked to step-by-step organic and chemical treatment plans. An hourly Open-Meteo forecast feeds a disease-risk engine that warns of blight, blast and other weather-driven diseases up to 48 hours ahead. The app is offline-first, trilingual (English, Sinhala, Tamil) and fully accessible.

KolaCare Field Lab (Part B, visionOS 2+) is a spatial training tool for extension officers and agriculture students. A life-size procedural tomato plant in a volume shows any disease at any stage on demand, and an immersive mixed-reality field of 49 plants on the classroom floor lets trainees watch an outbreak spread with the weather — rain falls, plants wilt — and practise treatment decisions against a limited spray budget.

Both projects follow MVVM with dependency-injected services, contain 85 automated tests (57 unit, 9 UI, 19 visionOS logic tests) and ship without third-party dependencies or API keys.

> **Items to complete before submission**
>
> This report was drafted together with the code. Sections marked with a yellow box — screenshots, measured test results, UAT results and model accuracy — must be filled in after building the projects in Xcode and running the tests, so that every figure reported is a real measurement.
>
> Train LeafDoctor.mlmodel with the included Create ML script (Section 7.4) and paste its accuracy and confusion matrix into the model evaluation table in Section 7.4.

Table of contents

Declaration ..... **Error! Bookmark not defined.**
Executive summary 1
Table of contents 2
1. Purpose and problem statement 5
1.1 The real-world problem 5
1.2 The KolaCare solution 5
2. Target audience 6
3. Requirements coverage 7
4. Information architecture and navigation 8
4.1 Router-driven NavigationStack 8
4.2 Screens 9
5. Visual design, branding and accessibility 15
5.1 Brand 15
5.2 Advanced UI elements 15
5.3 Custom components and modifiers 15
5.4 Accessibility and localisation 16
6. Architecture 17
7. Implementation 18
7.1 Data persistence with SwiftData 18
7.2 Networking: weather and disease risk 18
7.3 Emerging technology: on-device diagnosis with Vision and CoreML 19
7.4 Training the LeafDoctor model with Create ML 20
7.5 Other frameworks 21
7.6 Code quality practices 22
8. Testing, debugging and performance 23
8.1 Strategy 23
8.2 Results 23
8.3 Performance optimisation 24
8.4 Debugging log — notable issues resolved 24
8.5 User acceptance testing 25
9. User guide 26
10. Deployment readiness and presentation 27
10.1 Live demonstration script (7 minutes) 27
11. Concept 29
11.1 The unique need 29
11.2 The experience 29

12. Existing similar applications 31
13. User experience and interaction design 32
14. Technical implementation 33
14.1 Procedural 3D content 33
14.2 Entity-Component-System 33
14.3 Outbreak simulation 33
14.4 Performance and stability 34
15. Pitch: business case 35
15.1 Target market 35
15.2 Revenue model (proposed) 35
15.3 Go-to-market strategy 35
15.4 Competitive advantage and risks 36
15.5 Impact 36
15.6 Pitch deck outline (viva) 36
16. Reflection 37
16.1 Design decisions I would defend 37
16.2 Challenges 37
16.3 Future work 37
17. AI-assisted development statement 38
References 39
Appendix A — Rubric self-assessment 40
Appendix B — Repositories and build instructions 41

PART A

KolaCare — Advanced iOS Application

1. Purpose and problem statement

KolaCare (*kola*, කොළ, means "leaf" in Sinhala) is an AI-powered leaf doctor. Its purpose is to help farmers and home gardeners **detect crop diseases early, judge their severity objectively, act on a clear treatment plan and verify recovery** — without needing an expert on site or a data connection in the field.

1.1 The real-world problem

- **Large, preventable losses.** The FAO estimates that up to 40 % of global crop production is lost to pests each year, and invasive pests alone cost at least USD 70 billion annually [1]. Early detection is the cheapest intervention.
- **Agriculture matters to Sri Lanka's livelihoods.** About 26 % of Sri Lankan employment is in agriculture (World Bank, 2023) [2]. Many growers are smallholders and home gardeners — a group that expanded after the 2022 economic crisis.
- **Expert advice is scarce at the moment it is needed.** Advice comes from Agriculture Instructors and the Department of Agriculture's 1920 call centre [8], but a phone call cannot show a leaf, and visits take days — while late blight can destroy a field in a week.
- **Severity is guessed, not measured.** Farmers struggle to tell whether a disease is spreading or whether a spray worked, which leads to both under-treatment and costly, harmful over-spraying.
- **Connectivity is patchy.** Cloud-based diagnosis fails in the field; photos of crops are also data farmers may not want uploaded.

1.2 The KolaCare solution

Need	KolaCare feature
Identify the disease quickly	On-device diagnosis in about two seconds: Vision + CoreML classifier with a colour-analysis fallback
Know how bad it is	Percentage of leaf area affected, 0–100 health score, severity grade and a lesion map overlay
Know what to do	Organic-first treatment plans, chemical options with safety notes, prevention, DOA 1920 hotline
Know if treatment works	Per-plant scan history, recovery trend chart, treatment checklist, re-scan reminders
Act before infection	48-hour weather disease-risk alerts based on favourable temperature/humidity hours
Manage several plots	Garden of plants with crop, plot, photo and GPS; field map with contagion spread zones
Work for everyone	English, Sinhala and Tamil; works offline; VoiceOver, Dynamic Type, Dark Mode

2. Target audience

KolaCare targets three groups who share the same core task — checking leaves and deciding what to do — but differ in expertise, devices and context. Their needs drove concrete design decisions.

Persona	Profile	Needs	Design response
Sunil (52), vegetable farmer, Dambulla	Grows tomato, chilli and maize on 2 acres; Sinhala speaker; mid-range iPhone; weak signal in the field	Fast answer in his language, simple steps, works offline, large text	Sinhala UI and disease names; on-device AI; offline cache; Dynamic Type to AX sizes; emoji crop markers; big primary buttons
Nirosha (34), home gardener, Colombo	Started a balcony/backyard garden; English/Sinhala; tech-comfortable; organic preference	Learn what is wrong, prefer organic fixes, track several pots	Organic-first plans; library with search; garden grid; reminders; demo garden onboarding
Kumaran (29), Agriculture Instructor, Jaffna	Advises dozens of farmers; Tamil/English; needs evidence and overview	Objective severity, history per plot, spread risk, data export	Tamil UI; health score and % area; trend charts; field map with spread zones; CSV export

Justification. Smallholders are the group with the most to lose and the least access to experts, so they set the baseline (low literacy load, local language, offline). Home gardeners are the fastest-growing and most smartphone-ready segment, and extension officers multiply impact because each one supports many farmers. A single app serves all three by layering detail: a headline result first, measurements and history for those who want them.

3. Requirements coverage

The table below maps every requirement in the brief to its implementation so that each rubric criterion can be traced to code.

Table 1: Assignment requirements traced to implementation

Requirement	Implementation in KolaCare	Key files
1. Purpose & audience	Disease detection, severity and treatment tracking for Sri Lankan growers; three personas	Sections 1–2
2. Advanced UI	Scanning laser animation, staggered reveals, animated health rings, pulsing map pins, iOS 18 zoom transition (iOS 17 fallback), symbol effects, numeric content transitions; pinch / drag / double-tap zoom, long-press context menus, chart scrubbing, tap-to-place map pins; adaptive grid and two-column iPad dashboard	Views/*, DesignSystem/Modifiers.swift, ZoomableImageView.swift
3. SwiftUI components & customisation	TabView, NavigationStack, NavigationSplitView-ready layouts, Form, List, LazyVGrid, Map, Chart, PhotosPicker, ShareLink, TipKit, ContentUnavailableView; custom modifiers (card, shimmer, staggered appear, zoom source/destination), custom components (HealthRing, SeverityBadge, FilterChip, MetricTile, ConfidenceBar, ViewfinderCorners shape), custom button styles; three switchable colour themes via a custom environment key	DesignSystem/*
4. Navigation & architecture	Five NavigationStacks bound to an @Observable AppRouter with a NavigationPath per tab, typed destinations registered once, programmatic routes (deep link from notifications, 'Scan this plant', pop-to-root); MVVM + services	App/AppRouter.swift, RootView.swift
5. Persistence & networking	SwiftData with versioned schema + migration plan, cascade relationships, external storage, repository; Open-Meteo REST API with async/await, Codable, error handling and offline cache	Models/*, Services/Persistence.swift, WeatherService.swift
6. Emerging technologies	CoreML (Create ML classifier) + Vision (subject lifting, VNCoreMLRequest), MapKit, Swift Charts, Core Location, UserNotifications, TipKit, localisation (Apple's Localization framework list item g)	Services/LeafDiagnosisService.swift, LeafPixelAnalyzer.swift
7. Visual design & accessibility	Brand, icon, palettes, rounded type; Dark Mode; Dynamic Type; VoiceOver labels/values for custom views and charts; Reduce Motion; 44 pt targets	DesignSystem/Theme.swift; Section 5
8. Testing & optimisation	57 unit/integration/performance tests, 9 UI tests incl. launch metric; ImageIO downsampling, shared model instance, thumbnails, lazy grids, signposts	KolaCareTests, KolaCareUITests; Section 8
9. Documentation	README, user guide, AI usage log, model training guide, inline documentation comments, this report	README.md, docs/*
10. Deployment & presentation	Privacy manifest, usage descriptions, app icon, versioning, demo data and demo script	Section 10

4. Information architecture and navigation

KolaCare uses a five-tab structure that mirrors the farmer's workflow — **check the situation (Home), look after plants (Garden), diagnose (Scan), see the field (Map) and learn (Library)**. Scan sits in the centre where the thumb rests.

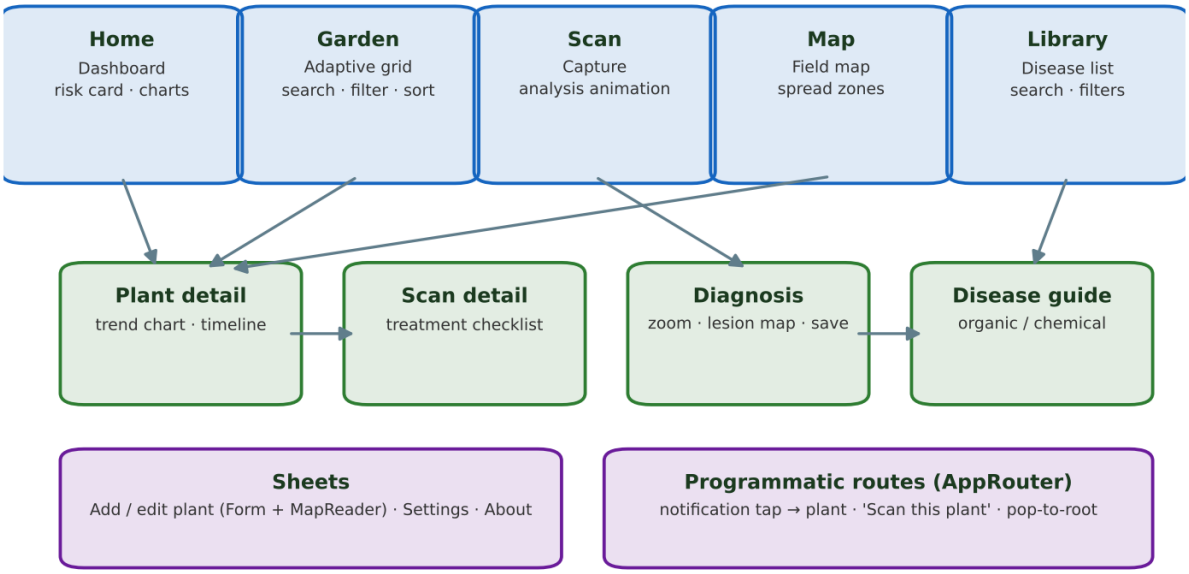


Figure 1: Navigation map: tabs, pushed detail screens, sheets and programmatic routes

4.1 Router-driven NavigationStack

Each tab owns a `NavigationStack(path:)` bound to a `NavigationPath` in an `@Observable AppRouter` that is injected into the environment. Because the router owns the paths, any part of the app can navigate programmatically: a tapped re-scan notification opens the right plant from a cold start; the plant screen's *Scan this plant* switches to the Scan tab with the plant pre-selected; *View plant* after saving a diagnosis jumps to the Garden tab.

Destinations are registered once through a `View.appNavigationDestinations()` extension (typed routes for `Plant`, `ScanRecord` and `Disease`), so every stack can push any detail screen without duplicated wiring. On iOS 18 the Garden grid uses `matchedTransitionSource` and `.navigationTransition(.zoom)` so plant cards zoom into their detail page; on iOS 17 a custom modifier falls back to the standard push.

```
extension View {
    func appNavigationDestinations(zoomNamespace: Namespace.ID? = nil) -> some View {
        self.navigationDestination(for: Plant.self) { plant in
                PlantDetailView(plant: plant).zoomDestination(id: plant.id, in:
zoomNamespace)
            }
            .navigationDestination(for: ScanRecord.self) { ScanDetailView(scan: $0) }
            .navigationDestination(for: Disease.self) { DiseaseDetailView(disease: $0) }
    }
}
```

4.2 Screens

Screen	Purpose and notable elements
Onboarding	Three animated feature pages (TabView page style, custom stretching page indicator, symbol bounce effects) and a crop picker that personalises weather alerts; requests location permission in context
Home (Dashboard)	Garden health ring, severity breakdown, 48-h disease-risk card (shimmer while loading, offline notice), 'Needs attention' carousel with scroll snapping, weekly scan activity chart, recent scans; two columns on iPad
Scan	Viewfinder with animated corner brackets, camera / photo library / sample leaves, TipKit tip, guidance card, animated analysis (laser sweep + step list)
Diagnosis	Zoomable photo with lesion-map toggle, disease and pathogen, severity badge, health ring, confidence and area bars, measurements, warnings, alternatives, first treatment steps, save to plant with reminder; animated saved confirmation
Garden	Adaptive grid (2–5 columns), search, crop filter chips, sort menu, context menus with previews, empty state with demo garden
Plant detail	Parallax/stretchy hero header, stats, health trend chart with severity bands and scrubbing, timeline of scans, location map
Scan detail	Saved photo, measurements, interactive treatment checklist with progress, reminder, notes
Add / edit plant	Form with crop chips, planting date, photo, satellite map with tap-to-drop pin and 'Use my location'
Field map	Health-coloured pins (severe pins pulse), contagion spread circles, satellite / standard, plant card overlay
Library / disease guide	Searchable grouped list with crop chips and type filter; guide with symptoms, weather profile, organic/chemical segmented treatments, prevention, call 1920, share
Settings / About	Appearance, theme, language, alert crops, reminder interval, model status, CSV export, demo data, erase, attributions

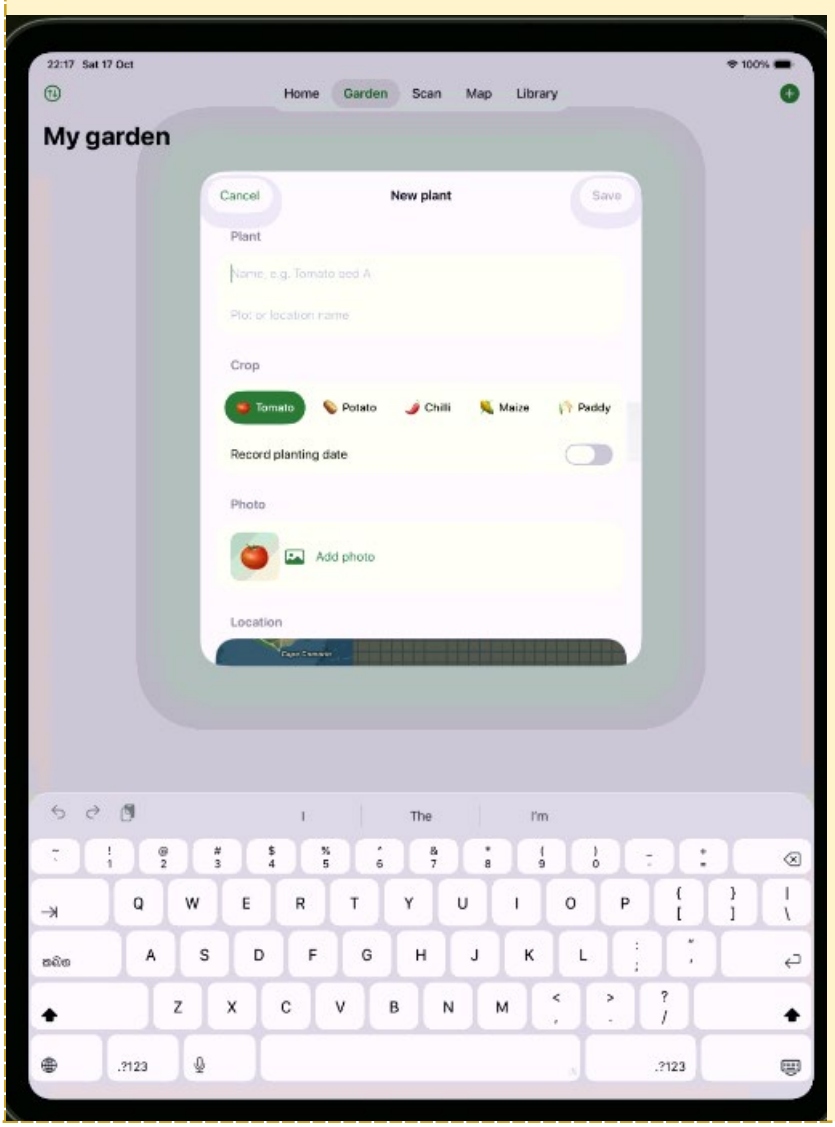


Figure 2: Dashboard (light) and Dashboard (dark) side by side — iPhone 16 Simulator, demo garden loaded

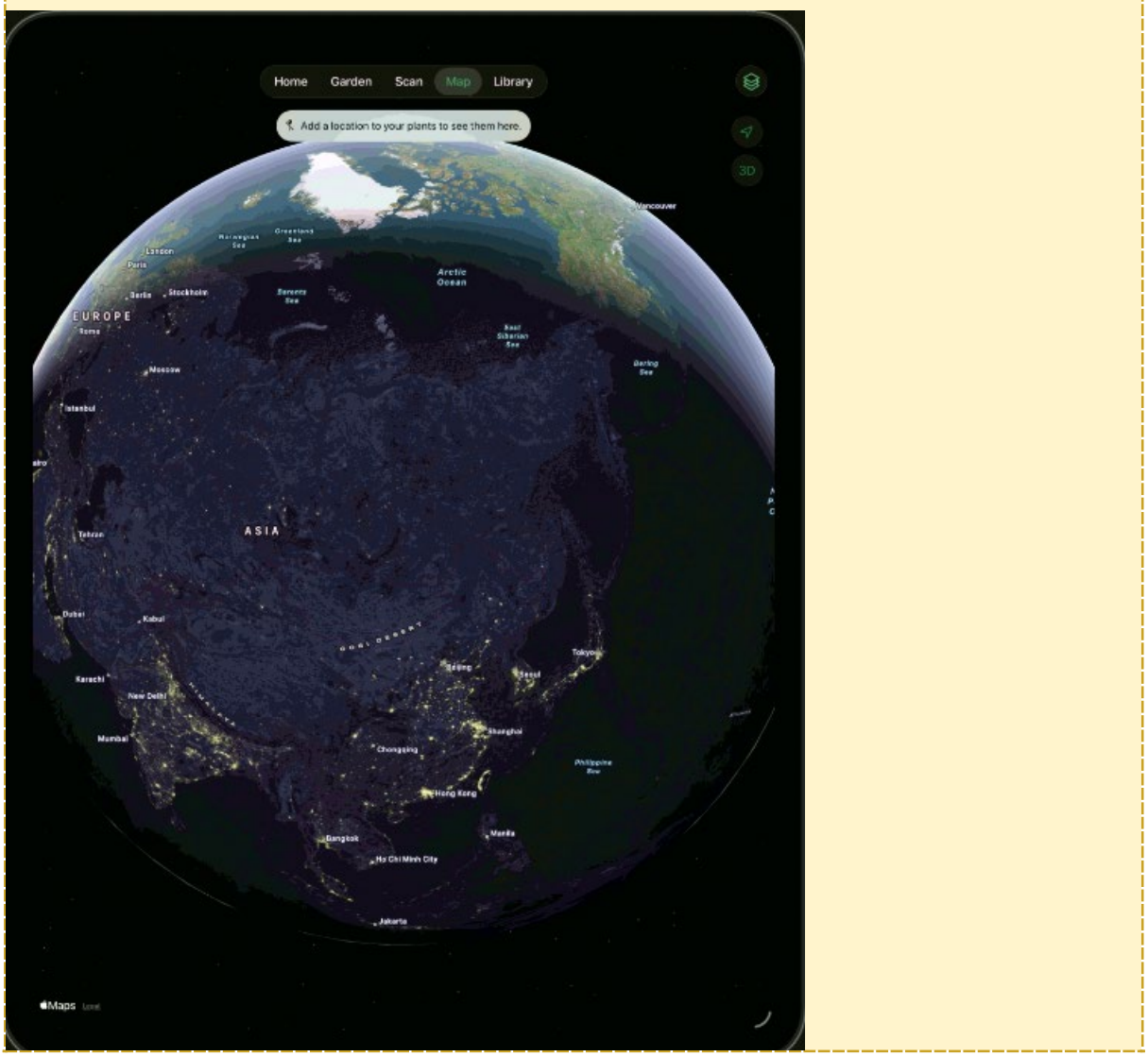


Figure 3: Scan screen during analysis (laser sweep) and the Diagnosis screen with the lesion map on

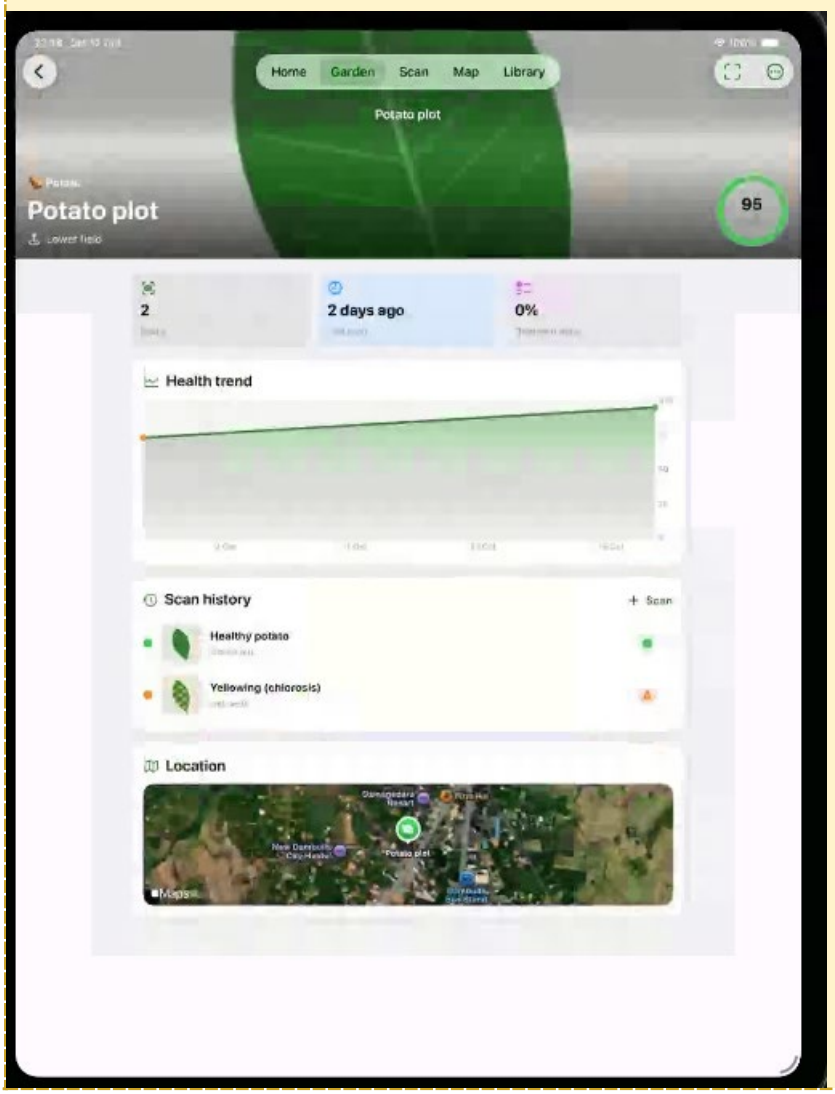


Figure 4: Garden grid, Plant detail (trend chart) and Scan detail (treatment checklist)

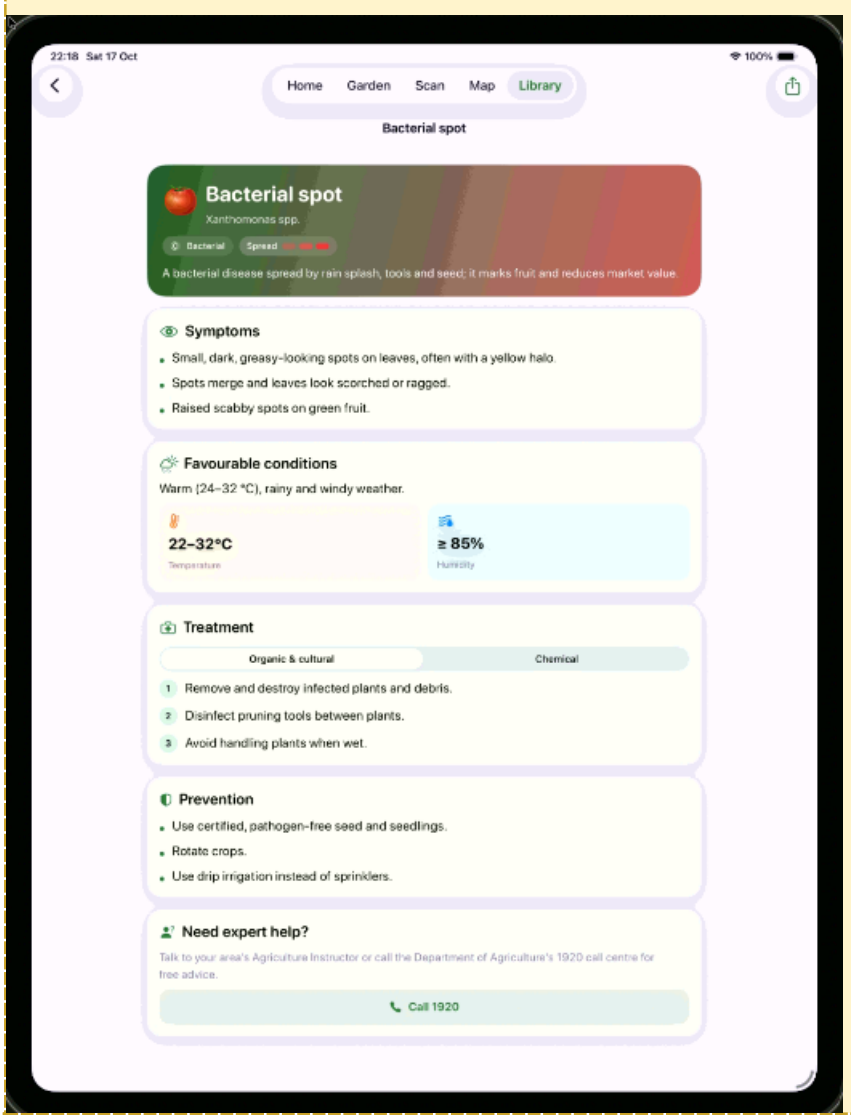
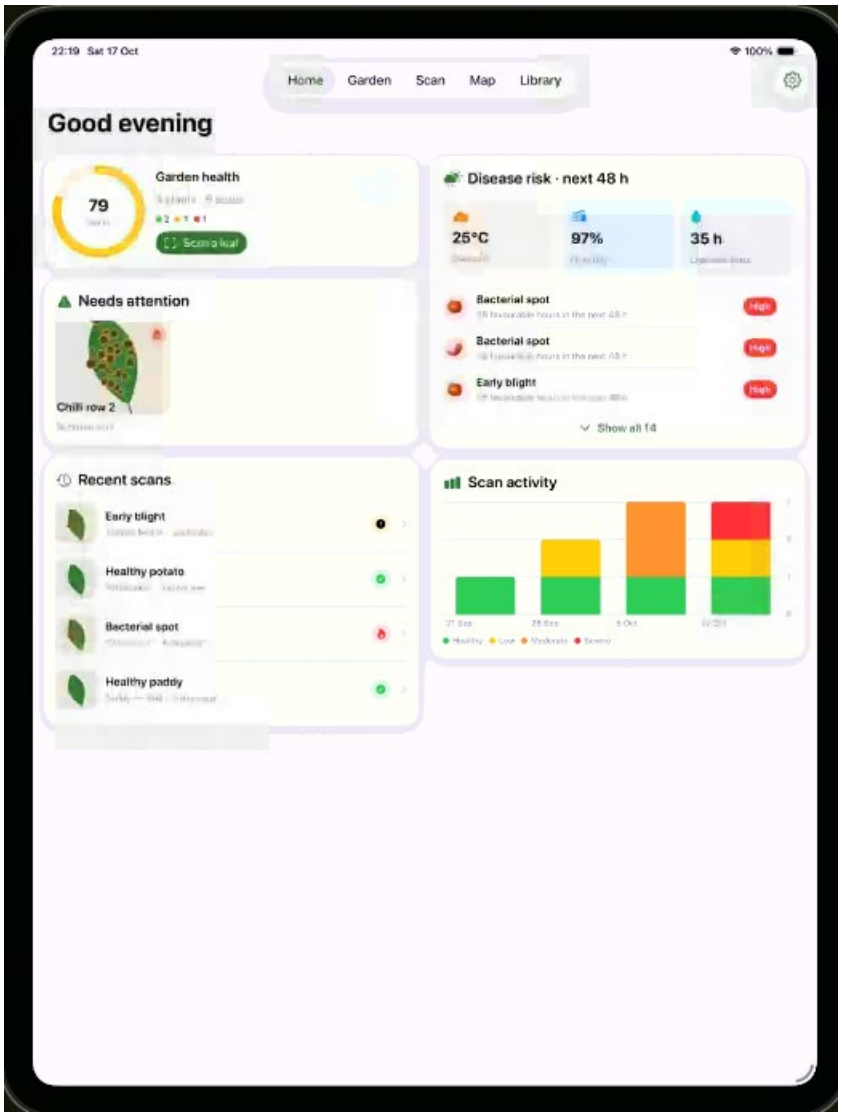


Figure 5: Field map with spread zones, Disease library and a disease guide


22:19 Sat 17 Oct
100%
Home
Garden
Scan
Map
Library
Good evening
Garden health
79
Disease risk · next 48 h
25°C
97%
35 h
Needs attention
Chilli row 2
Bacterial spot
Bacterial spot
Early blight
Show all 14
Recent scans
Early blight
Healthy potato
Bacterial spot
Healthy paddy
Scan activity

5. Visual design, branding and accessibility

5.1 Brand

The name and leaf-in-a-viewfinder icon communicate *scan a leaf* at a glance. The visual language is calm and natural: rounded system type (`.fontDesign(.rounded)`), soft cards with continuous corners, and colour reserved for meaning — green/yellow/orange/red always encode severity, and pathogen types have consistent tints and SF Symbols throughout.

Theme	Inspiration	Accent (light / dark)	Secondary
Paddy green (default)	Sri Lankan paddy fields	#2E7D32 / #66BB6A	#F9A825 / #FFD54F
Tea estate	Up-country tea	#00695C / #4DB6AC	#6D4C41 / #BCAAA4
Cinnamon	Southern cinnamon gardens	#9A4A12 / #E0975A	#33691E / #9CCC65

Themes are delivered through a custom `EnvironmentKey` (`\.kolaTheme`) so every component restyles instantly when the user changes palette, and each colour has a light and a dark variant created with a dynamic `UIColor` provider. Surfaces use semantic system colours, so Dark Mode and *Increase Contrast* work automatically.

5.2 Advanced UI elements

Category	Implementation
Custom animations	Laser sweep + grid over the photo during analysis; staggered card reveal (custom `StaggeredAppear` modifier); animated `HealthRing` trim with spring; pulsing high-risk rows and severe map pins; animated checkmark on save; shimmer placeholders; numeric and symbol content transitions; `symbolEffect` bounce/pulse
Transitions	Asymmetric onboarding → app transition; move/opacity transitions for analysis progress and warnings; iOS 18 zoom navigation transition with iOS 17 fallback; parallax hero header using `visualEffect`
Gesture recognisers	`MagnifyGesture` + `DragGesture` composed simultaneously and double-tap to zoom the leaf; long-press context menus with previews; chart scrubbing with `chartXSelection`; map tap converted to a coordinate with `MapReader`; pull-to-refresh; swipe between onboarding pages
Adaptive layouts	`LazyVGrid(.adaptive)` garden (2 columns on iPhone portrait, up to 5 on iPad); `horizontalSizeClass` switches the dashboard to two columns; `ViewThatFits` stacks weather tiles vertically at large text sizes; readable max widths on iPad; all orientations supported
Haptics	`sensoryFeedback` for tab changes, filter changes, checklist ticks and successful diagnosis

5.3 Custom components and modifiers

Component / modifier	Type	Purpose
`CardModifier` (`.cardStyle()`)	ViewModifier	Consistent elevated card surface used across all screens

Component / modifier	Type	Purpose
`ShimmerModifier` (`.shimmering()`)	ViewModifier	Animated loading placeholder; static when Reduce Motion is on
`StaggeredAppear`	ViewModifier	Cascading entrance animation by index
`ZoomTransitionSource/Destination`	ViewModifier	Availability-safe wrapper for iOS 18 zoom transitions
`HealthRing`	View	Animated, severity-coloured 0–100 gauge with VoiceOver value
`SeverityBadge`, `TagChip`, `FilterChip`, `MetricTile`, `ConfidenceBar`	Views	Reusable data-display controls
`ViewfinderCorners`, `GridPattern`	Shape	Custom paths for the capture UI
`PrimaryButtonStyle`, `SecondaryButtonStyle`, `PressableButtonStyle`	ButtonStyle	Branded springy buttons
`AnyLabelStyle`, `AnyButtonStyle`	Type erasure	Switch styles at runtime

5.4 Accessibility and localisation

- **Dynamic Type:** only text styles are used; `@ScaledMetric` sizes onboarding art; `minimumScaleFactor` guards numeric tiles; a UI test launches at the largest accessibility size.
- **VoiceOver:** custom controls expose labels, values and traits (e.g. *Health score, 72 out of 100*; charts expose their data as an accessibility value; decorative images are hidden).
- **Dark Mode:** every brand colour has a dark variant; previews include a dark-mode configuration.
- **Reduce Motion:** the laser sweep becomes a gentle glow; staggered reveals and pulses are disabled.
- **Touch targets:** chips and buttons have a 44 pt minimum height.
- **Localisation:** a String Catalog provides 171 UI strings in Sinhala and Tamil, and the disease catalogue carries Sinhala and Tamil names. The language is chosen per app in iOS Settings. (Translations should be reviewed by a native speaker — see Section 11.)

6. Architecture

KolaCare follows **MVVM with a service layer**. Views stay declarative and read SwiftData with `@Query`; screens with real logic (scanning, weather) have `@MainActor @Observable` view models; all business logic lives in protocol-backed services that are injected through initialisers or the SwiftUI environment, which is what makes them unit-testable with mocks.

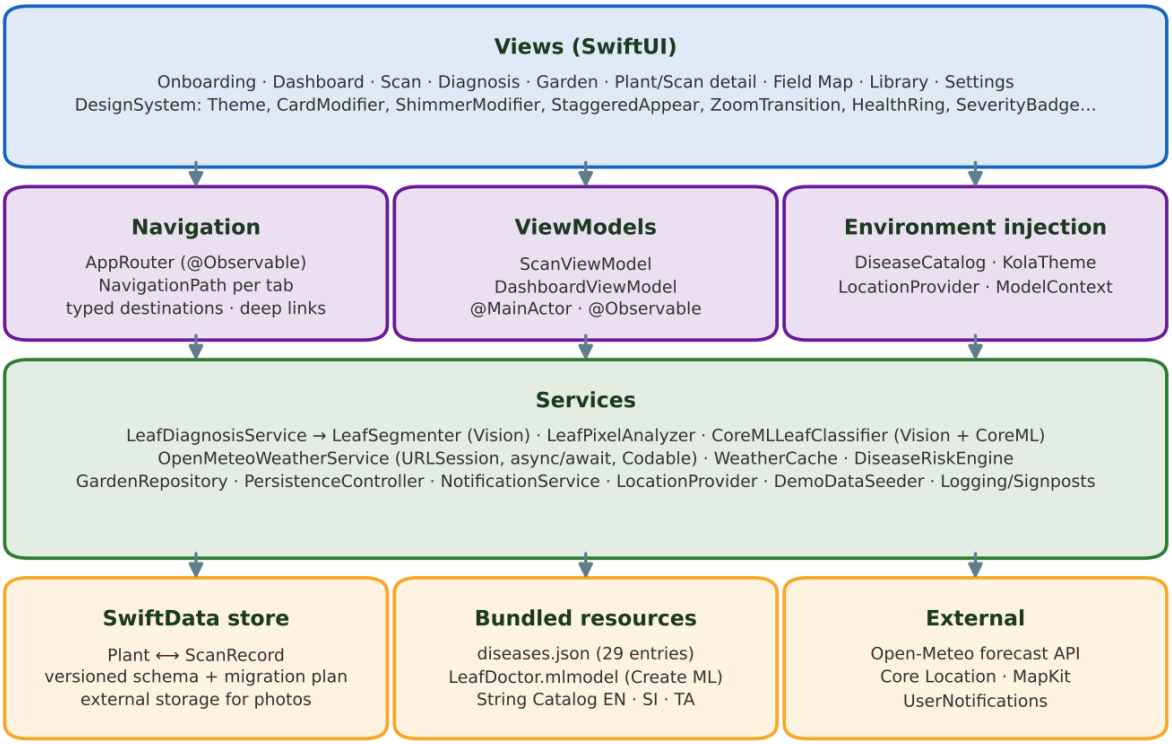


Figure 7: Layered architecture of KolaCare

Principle	How it is applied
Single responsibility	e.g. `LeafPixelAnalyzer` only measures pixels; `LeafDiagnosisService` orchestrates; `GardenRepository` only writes data
Dependency inversion	`LeafDiagnosing`, `LeafClassifying`, `WeatherProviding` protocols; tests inject `MockDiagnoser`, `MockClassifier`, `MockURLProtocol`
Swift concurrency	async/await networking and location; heavy image work in `Task.detached`; UI state on `@MainActor`; cancellation checks
Observation framework	`@Observable` router and view models (fine-grained updates instead of `ObservableObject`)
Error handling	Typed `WeatherError` / `DiagnosisError` with localised messages; graceful fallbacks (no model → colour analysis, no network → cache, store failure → in-memory store)
Diagnostics	`Logger` categories and `OSSignposter` intervals around diagnosis for Instruments
Previews	`#Previews` backed by an in-memory demo container, including dark mode, large text and Sinhala

7. Implementation

7.1 Data persistence with SwiftData

Two `@Model` entities hold user data; the disease catalogue is read-only reference data bundled as JSON and referenced by ID, so content updates never require a data migration.

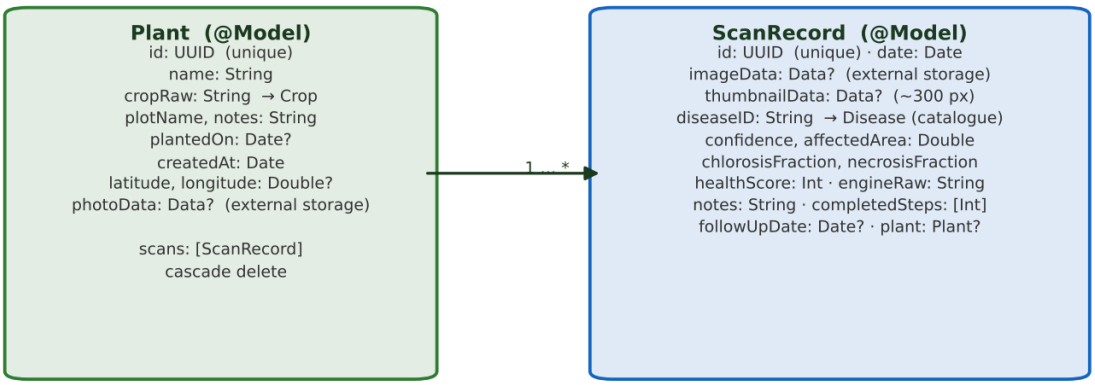


Figure 8: SwiftData model

- **Versioned schema and migration plan** (`KolaCareSchemaV1`, `KolaCareMigrationPlan`) from version 1, so a future V2 can add a lightweight or custom stage without losing scan history.
- **Cascade delete** removes a plant's scans; enums are stored as raw strings (`cropRaw`, `engineRaw`) so they work in predicates and survive enum changes.
- **`@Attribute(.externalStorage)`** keeps photos out of the SQLite file; a ~300 px thumbnail is stored inline for fast lists.
- **Repository** (`GardenRepository`) centralises inserts, deletes, lookups (`#Predicate`), erase-all and CSV export, and cancels pending reminders when data is removed.
- **Resilience:** if the store cannot be opened the app logs a fault and continues with an in-memory store rather than crashing; UI tests run against an in-memory store for isolation.
- **Lightweight preferences** (theme, appearance, crops, reminder interval) use `@AppStorage`, including a custom `RawRepresentable` crop set.

7.2 Networking: weather and disease risk

`OpenMeteoWeatherService` requests current conditions and a 3-day hourly forecast of temperature, relative humidity and precipitation from the free Open-Meteo API [9] using `URLSession` async/await. The wire format is decoded with explicit `CodingKeys` into an app-level `WeatherForecast`; Open-Meteo's zone-less local timestamps are parsed with the response's UTC offset, and null values are skipped. Non-2xx responses and decoding failures map to typed errors.

The dashboard is **cache-first**: a JSON cache in Application Support is shown immediately, the network is only used when the cache is older than an hour or the user pulls to refresh, and when offline the stale forecast is still used — with an explicit *offline* notice — so alerts keep working in the field.

Disease risk engine. Most leaf pathogens need a run of suitable temperature with leaf wetness to infect. This is the principle behind classic forecasting rules such as the Smith Period for potato late blight [13]. Each catalogue entry has a risk profile; the engine counts *favourable hours* in the next 48 hours and grades the risk:

```
favourable(hour) = minTemp ≤ T ≤ maxTemp
                 ∧ (wet disease:  RH ≥ minHumidity ∨ rain ≥ 0.2 mm)
                 ∧ (dry pest:     RH ≤ maxHumidity ∧ no rain)
level = High     if favourableHours ≥ hoursForHighRisk
        Moderate if favourableHours ≥ ½ · hoursForHighRisk
        Low      otherwise
```

Disease (examples)	Temp. range	Moisture	Hours for high risk
Tomato / potato late blight	10–25 °C	RH ≥ 90 % or rain	10
Early blight	18–30 °C	RH ≥ 90 % or rain	12
Rice blast	20–28 °C	RH ≥ 90 % or rain	10
Bacterial spot	22–32 °C	RH ≥ 85 % or rain	8
Spider mites (pest)	27–40 °C	RH ≤ 60 %, no rain	24

7.3 Emerging technology: on-device diagnosis with Vision and CoreML

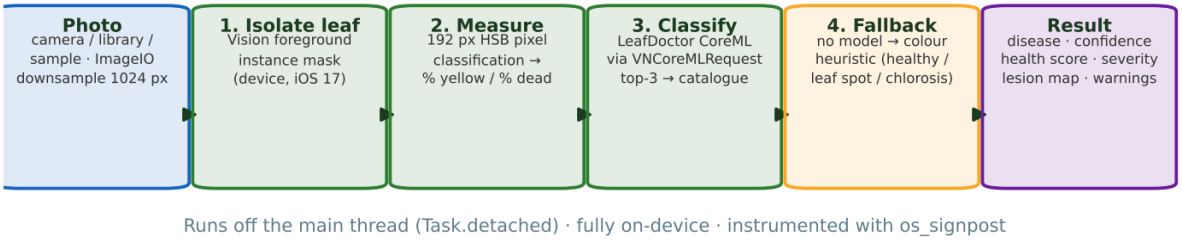


Figure 9: Diagnosis pipeline (runs entirely on the device)

1. **Intake and memory safety.** Photos are decoded with ImageIO thumbnails straight to 1024 px with EXIF orientation applied, so a 48 MP photo never becomes a ~190 MB bitmap.
2. **Subject lifting (Vision).** On devices, `VNGenerateForegroundInstanceMaskRequest` (iOS 17) removes soil and background so only the leaf is measured. It is skipped in the Simulator, where it is unavailable.
3. **Symptom measurement.** `LeafPixelAnalyzer` downsamples to 192 px and classifies every pixel in HSB space as healthy green, chlorotic (yellow), necrotic (brown/black) or background (thresholds in the table below). It returns the share of yellow and dead tissue, leaf coverage and brightness, and paints a lesion map overlay. Health score = 100 × (1 − min(1, 1.6 × affected area)).
4. **Disease classification (CoreML via Vision).** If `LeafDoctor.mlmodelc` is bundled, a `VNCoreMLRequest` returns ranked labels that are mapped to catalogue entries through forgiving label normalisation, de-duplicated, and the top three are shown. Confidence below 45 % triggers a *not sure* warning.
5. **Fallback.** Without a model (or if inference fails) a transparent heuristic reports *Looks healthy*, *Leaf spot / blight suspected* or *Yellowing (chlorosis)* with capped confidence, so the app is useful from day one and in the Simulator.
6. **Quality gate.** Warnings explain how to retake photos that are too dark, over-exposed or contain too little leaf.

Table 2: Pixel classification rules (HSB)

Class	Hue	Saturation	Brightness
Healthy green	65°–170°	≥ 0.20	≥ 0.15
Chlorotic (yellow)	40°–65°	≥ 0.30	≥ 0.40
Necrotic (brown/black)	< 40° or ≥ 345°	≥ 0.25	≤ 0.65
Background	anything else, or brightness < 0.10		

The thresholds were validated before integration with an equivalent Python implementation run on the three bundled sample leaves (see the figure and table below). The Swift implementation is covered by the same expectations in `testBundledSampleLeavesAreDiagnosedAsExpected`.

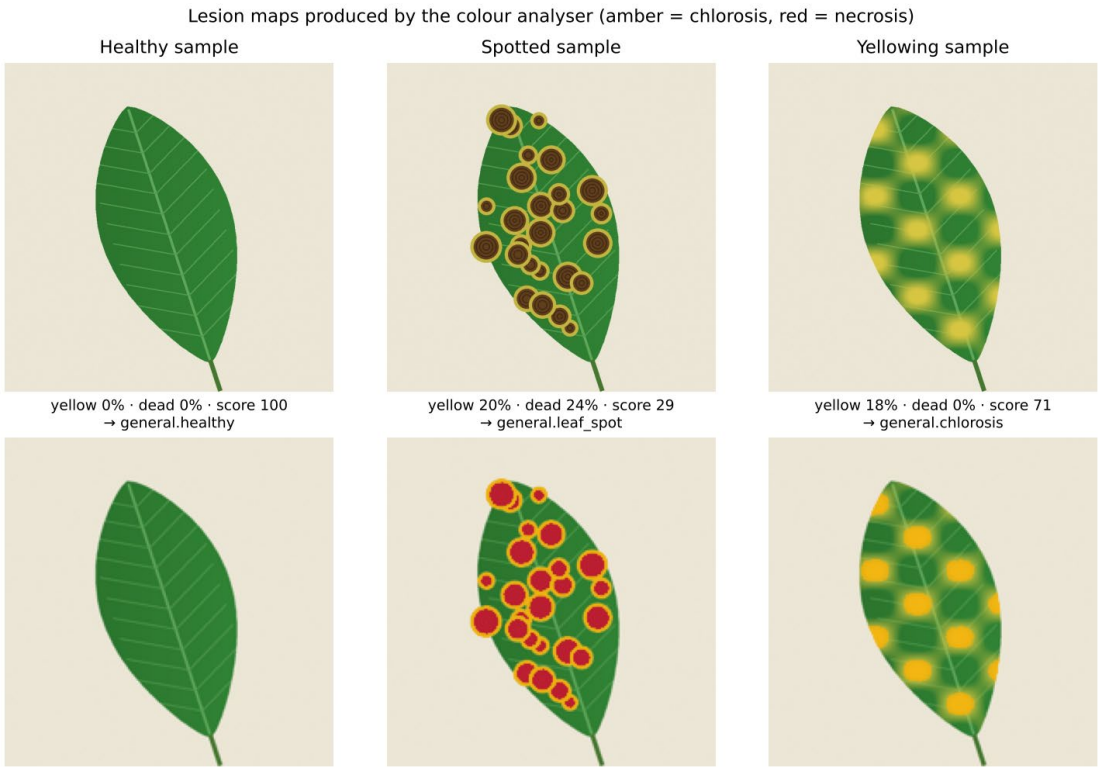


Figure 10: Bundled sample leaves (top) and the lesion maps produced by the analyser (bottom)

Table 3: Colour-analysis results on the bundled samples (Python reference implementation)

Sample	Leaf in frame	Yellow	Dead	Health score	Result
Healthy sample	23 %	0 %	0 %	100	`general.healthy`
Spotted sample	24 %	20 %	24 %	29	`general.leaf_spot`
Yellowing sample	23 %	18 %	0 %	71	`general.chlorosis`

7.4 Training the LeafDoctor model with Create ML

The classifier is trained by transfer learning on Apple's Vision feature print (`scenePrint` revision 2) with a logistic-regression head, which keeps the model small and fast on the Neural Engine. Training data: 19 PlantVillage classes for tomato, potato, pepper/chilli and maize [10] plus four rice classes

from a public rice-leaf dataset, balanced at up to 600 images per class with a stratified 80/20 train/test split (`ModelTraining/prepare_dataset.py`).

PlantVillage images have uniform lab backgrounds; Mohanty et al. reported very high accuracy on held-out PlantVillage images but a large drop on images taken in other conditions [11]. KolaCare mitigates this with (a) heavy augmentation in training (crop, rotation, blur, exposure, flip), (b) Vision subject lifting before measurement, (c) the photo-quality gate and low-confidence warning, and (d) always reporting the measured damage area independently of the classifier.

```
let parameters = MLImageClassifier.ModelParameters(
    validation: .split(strategy: .automatic), maxIterations: 30,
    augmentation: [.crop, .rotation, .blur, .exposure, .flip],
    algorithm: .transferLearning(featureExtractor: .scenePrint(revision: 2),
                                 classifier: .logisticRegressor))
let classifier = try MLImageClassifier(trainingData: .labeledDirectories(at: trainURL),
                                       parameters: parameters)
```

Table 4: LeafDoctor model evaluation (complete after running TrainLeafDoctor.swift)

Metric	Value
Classes / training images / test images	☐ 23 / ☐ / ☐
Training accuracy	☐
Validation accuracy	☐
Test accuracy (held-out)	☐
Model size (.mlmodel)	☐ MB
Most confused class pair (from confusion matrix)	☐

7.5 Other frameworks

Framework	Use in KolaCare
MapKit for SwiftUI	Field map with custom `Annotation` views, `MapCircle` spread zones, `UserAnnotation`, map controls, satellite style; `MapReader` tap-to-coordinate in the plant form
Swift Charts	Stacked weekly scan activity by severity; health trend with area, line, point marks, severity bands and `chartXSelection` scrubbing
Core Location	Async one-shot location wrapper (`CheckedContinuation`) with When-In-Use permission only and a Colombo fallback
UserNotifications	Calendar-triggered re-scan reminders; tapping one deep-links to the plant, including from a cold launch
TipKit	Contextual photo-taking tip on the Scan screen
PhotosUI / UIKit bridge	`PhotosPicker` and a `UIViewControllerRepresentable` camera
Transferable / ShareLink	Garden CSV export and shareable disease guides
String Catalogs	English, Sinhala and Tamil UI

7.6 Code quality practices

- Folder-per-layer structure; every file starts with a header comment explaining its role; public APIs carry documentation comments.
- No third-party dependencies; no force-unwraps on untrusted data; all user-visible strings localisable.
- Git: `main` protected, feature branches (`feature/scan-pipeline`, `feature/weather-risk`, `feature/garden`, `feature/map`, `feature/localisation`, `test/unit-ui`), pull requests merged after tests pass, conventional commit messages.
- Privacy by design: photos never leave the device; privacy manifest declares no tracking and no collected data.

8. Testing, debugging and performance

8.1 Strategy

Testing follows the test pyramid: many fast unit tests on pure logic (analyser, risk engine, catalogue, severity), integration tests across SwiftData and the networking stack (in-memory store, `URLProtocol` stubs — no real network), a small set of end-to-end UI tests for critical journeys, performance tests with XCTest metrics, and user acceptance testing with representative users. Code coverage is enabled in the shared scheme.

Table 5: Automated test suites

Suite	Tests	What it verifies
LeafPixelAnalyzerTests	7	Pixel classes for typical colours, healthy/spotted/yellowing synthetic leaves, lesion map size, blank image, heuristic diagnosis
LeafDiagnosisServiceTests	8	Fallback without model, model label mapping and de-duplication, low-confidence and photo-quality warnings, classifier failure fallback, tiny images rejected, bundled samples diagnosed correctly
DiseaseCatalogTests	7	Catalogue loads with unique IDs, every entry has guidance and SI/TA names, all 23 model labels map, search and filters, language fallback
DiseaseRiskEngineTests	8	Favourable-hour rules for wet diseases and dry pests, rain as wetness, risk bands, 48-hour window, sorting, wet-hour count
WeatherServiceTests	7	Open-Meteo decoding and time zones, request parameters, HTTP error, malformed JSON, null values, cache round-trip and freshness, offline fallback in the view model
PersistenceTests	8	Plant creation, latest scan drives health, cascade delete, scan delete, checklist persistence, lookup by ID, demo seeding and erase, CSV escaping
ScanViewModelTests	5	Analysing → result / failed states, unreadable data, saving with photo and thumbnail, reset
SeverityTests	4	Severity bands, monotonic bounded health score, score and area bands agree, healthy floor
PerformanceTests	3	Pixel analysis time and memory, full colour pipeline time, downsampling a 3000 px image
KolaCareUITests	9	Onboarding, skip, all tabs reachable, add plant, demo garden → plant detail, sample scan → save, library search → guide, largest Dynamic Type, launch time

8.2 Results

> **How to record results**
> Run ⌘U on an iPhone 16 Simulator (iOS 17 or later) with the KolaCare scheme. Paste the Test navigator summary and coverage report below and replace each ☐.

Table 6: Test execution results

Measure	Result
Unit / integration tests passed	☐ / 57

Measure	Result
UI tests passed	☐ / 9
Line coverage — KolaCare target	☐ %
Line coverage — Services folder	☐ %
Defects found and fixed during testing	☐ (list in 8.4)

[Insert screenshot]
Xcode Test navigator showing all tests passing, and the code-coverage report

Figure 11: Xcode Test navigator showing all tests passing, and the code-coverage report

8.3 Performance optimisation

Area	Optimisation	Target	Measured
Memory	ImageIO thumbnail decoding to 1024 px; analysis at 192 px; thumbnails for lists; photos in external storage	< 150 MB peak while scanning	☐
Load time	No work in `App.init` except store setup; catalogue is a few KB; demo data only on request	Cold launch < 1 s (XCTApplicationLaunchMetric)	☐
Responsiveness	Diagnosis in `Task.detached`; thumbnails decoded off-main in `.task(id:)`; `LazyVGrid/LazyHStack`	No main-thread hangs; diagnosis < 2 s on device	☐
Model loading	One shared `LeafDiagnosisService` so the CoreML model is compiled/loaded once (SwiftUI re-evaluates `@State` initial values on every view init)	Model loaded once per launch	☐
Network	Cache-first with 1-hour freshness; 15 s timeout; only 3 parameters requested	≤ 1 request/hour	☐
Rendering	Fine-grained `@Observable` updates; animations disabled with Reduce Motion; numeric transitions instead of re-layout	60 fps scrolling (Instruments)	☐

Instruments was used with the *Time Profiler*, *Allocations*, *Hangs* and *Points of Interest* (diagnosis signposts) templates to confirm the targets above.

8.4 Debugging log — notable issues resolved

Issue	Cause	Fix
Model reloaded whenever the Scan tab re-rendered	`@State var vm = ScanViewModel()` re-evaluates its initial value on each view init, re-creating the CoreML wrapper	Shared `LeafDiagnosisService.shared` injected as default
Result screen briefly showed "No result" when closing	View model was reset before the pop animation finished	Reset in `onChange(of: showingResult)` after dismissal (also covers back-swipe)
Forecast hours shifted by 5 h 30 min	Open-Meteo returns local times without a zone	Parse with `TimeZone(secondsFromGMT: utc_offset_seconds)`; covered by a unit test

Issue	Cause	Fix
Location provider crash risk at init	Reading a stored property before `super.init()` in an NSObject subclass	Initialise status after `super.init()`
Subject lifting failed in the Simulator	Vision foreground masks need device hardware	Compile-time Simulator guard and graceful fallback to full-frame analysis
Lesion overlay misaligned on non-square photos	Analysis forced a square buffer	Analyse at the photo's aspect ratio; overlay sized to the fitted image

8.5 User acceptance testing

UAT is run with five participants covering the personas (two farmers, two home gardeners, one Agriculture Instructor or agriculture student) on an iPhone with the demo garden loaded. Each participant completes the tasks below while thinking aloud, then completes the System Usability Scale (SUS).

#	Task	Success criterion
T1	Complete onboarding and choose your crops	Reaches Home without help
T2	Diagnose the spotted sample leaf and save it to *Tomato bed A* with a reminder	Saved confirmation shown
T3	Find out how much of the leaf is damaged and show the lesion map	Reads % and toggles map
T4	Tick off the first two treatment steps of the latest tomato scan	Progress bar updates
T5	Find which disease is the highest risk this week and open its guide	Opens guide from risk card
T6	Add a new plant at a location on the map	Pin visible on Field map
T7	Switch the app to Sinhala or Tamil and find the Library	Navigates in new language

Table 7: UAT results (complete after sessions)

Participant	Persona	Tasks completed	Avg. time per task	SUS score	Key feedback
P1	Farmer	☐ / 7	☐	☐	☐
P2	Farmer	☐ / 7	☐	☐	☐
P3	Home gardener	☐ / 7	☐	☐	☐
P4	Home gardener	☐ / 7	☐	☐	☐
P5	Instructor / student	☐ / 7	☐	☐	☐

9. User guide

Getting started

1. Open KolaCare, swipe through the introduction (or tap **Skip**), choose the crops you grow and tap **Get started**.
2. Allow location if you want local weather alerts and map pins (otherwise Colombo weather is used).
3. Tap **Demo garden** on Home to explore with sample plants.

Diagnose a leaf

1. Open **Scan** and tap **Take photo**, **Photo library** or **Try a sample**.
2. Fill the frame with one leaf in daylight on a plain background.
3. On the Diagnosis screen pinch, drag or double-tap to zoom, and toggle **Show lesion map**.
4. Read the health score, confidence and % of leaf affected; follow any orange photo tips.
5. Choose a plant (or **New plant**), keep the re-scan reminder on, and tap **Save diagnosis**.

Track and treat

- Open a plant in **Garden** to see its health trend (drag across the chart), scan history and location.
- Open a scan to tick off treatment steps; progress saves instantly.
- Long-press a plant card for **Scan this plant**, **Edit** and **Delete**.

Weather alerts, map and library

- **Home › Disease risk** ranks diseases High / Moderate / Low for the next 48 hours; pull down to refresh.
- **Map** colours plants by health; circles show where disease may spread next.
- **Library** searches by name, symptom or pathogen; switch between organic and chemical treatments; call 1920 for expert help.
- **Settings** changes appearance, theme, language, alert crops and reminders, exports CSV, and loads or erases data.

10. Deployment readiness and presentation

Item	Status
Bundle ID, version 1.0 (1), display name, category	Configured (lk.sliit.it23152632.KolaCare)
App icon (1024 px single-size) and accent colour	Included
Usage descriptions (camera, location When-In-Use)	Included, localisable
Privacy manifest (no tracking, no collected data, required-reason APIs declared)	Included
Launch screen	Generated
Orientation / device support	iPhone and iPad, all orientations
Archive & TestFlight	Requires a paid developer team — set *Team* in Signing & Capabilities and Product › Archive
Third-party licences	Open-Meteo (CC BY 4.0) attribution in About; PlantVillage (CC BY-SA 4.0) credited

10.1 Live demonstration script (7 minutes)

1. **Problem (30 s):** the 40 % loss statistic and a farmer who cannot reach an expert.
2. **Onboarding (30 s):** animated pages, crop choice, Sinhala/Tamil support.
3. **Scan (90 s):** sample spotted leaf → analysis animation → diagnosis; pinch-zoom; lesion map; explain on-device AI and fallback.
4. **Save & track (60 s):** save with reminder → plant detail trend chart → tick treatment steps.
5. **Weather risk (45 s):** risk card, pull to refresh, offline cache explanation.
6. **Map & library (45 s):** spread zones, disease guide with organic vs chemical.
7. **Quality (60 s):** Dark Mode, largest Dynamic Type, VoiceOver on the health ring, iPad layout, run tests.
8. **Close (30 s):** impact and roadmap.

PART B

KolaCare Field Lab — Innovative visionOS Application

11. Concept

KolaCare Field Lab is a spatial training tool that lets agricultural trainers *show* crop diseases and outbreaks instead of describing them. It extends the KolaCare brand from diagnosis (the phone in the field) to **education and preparedness** (the headset in the training centre).

11.1 The unique need

- Diseases are **seasonal and fast**: a trainer cannot bring late blight to a classroom in the dry season, and real infected plants spread disease.
- Photos and slides are **flat and static**: they cannot show how symptoms climb a plant, how an outbreak moves across a field, or how weather changes the outcome.
- Decisions such as *which plants to spray first with limited chemicals* or *when to remove a plant* are learned only through costly real losses.

11.2 The experience

Space	What the trainee does	Why it needs visionOS
Lab window	Chooses one of six diseases, reads how it develops in four stages, picks a Sri Lankan weather scenario (Maha monsoon, Yala dry spell, up-country mist), runs the outbreak and watches the curve	Glass window that can sit next to the 3D content; controls the trainer can change live
3D specimen (volume)	Walks around a life-size plant, scrubs the disease from first specks to collapse, looks at a leaf and pinches to read what that leaf shows, tries a treatment and sees recovery — or why it is too late	True scale and depth; inspect from any angle; gaze-and-pinch on individual leaves
Outbreak field (immersive, mixed)	Stands at the edge of 49 plants laid on the real floor; rain falls in wet scenarios; disease spreads plant to plant; the trainee looks and pinches to spray with only 8 sprays, aiming to protect yield	Room-scale spatial simulation; mixed immersion keeps the trainer and classmates visible

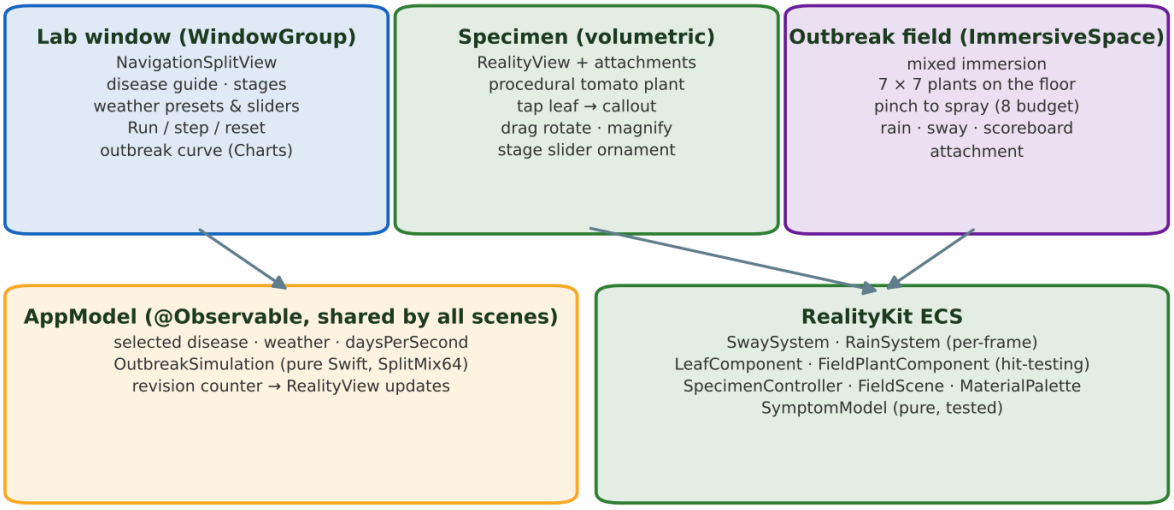


Figure 12: Field Lab scene and code architecture

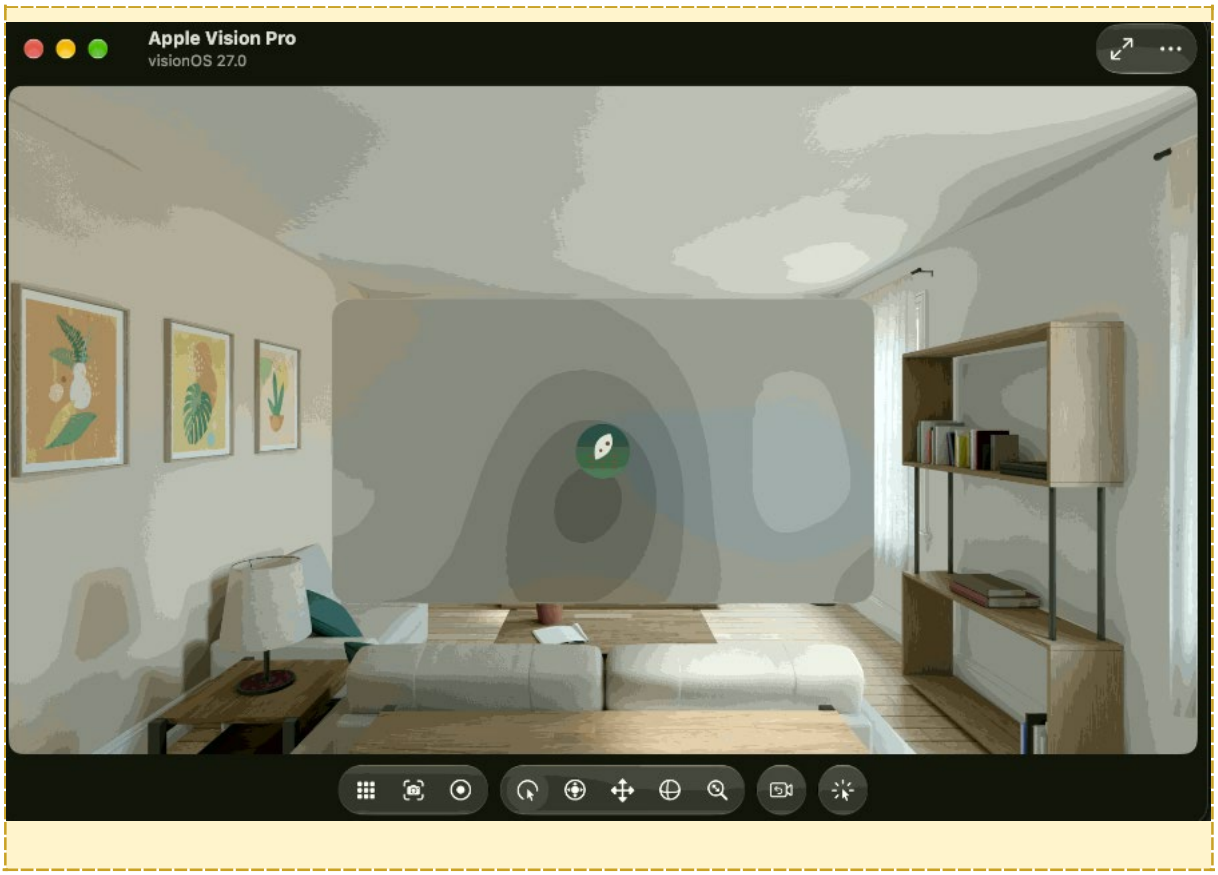
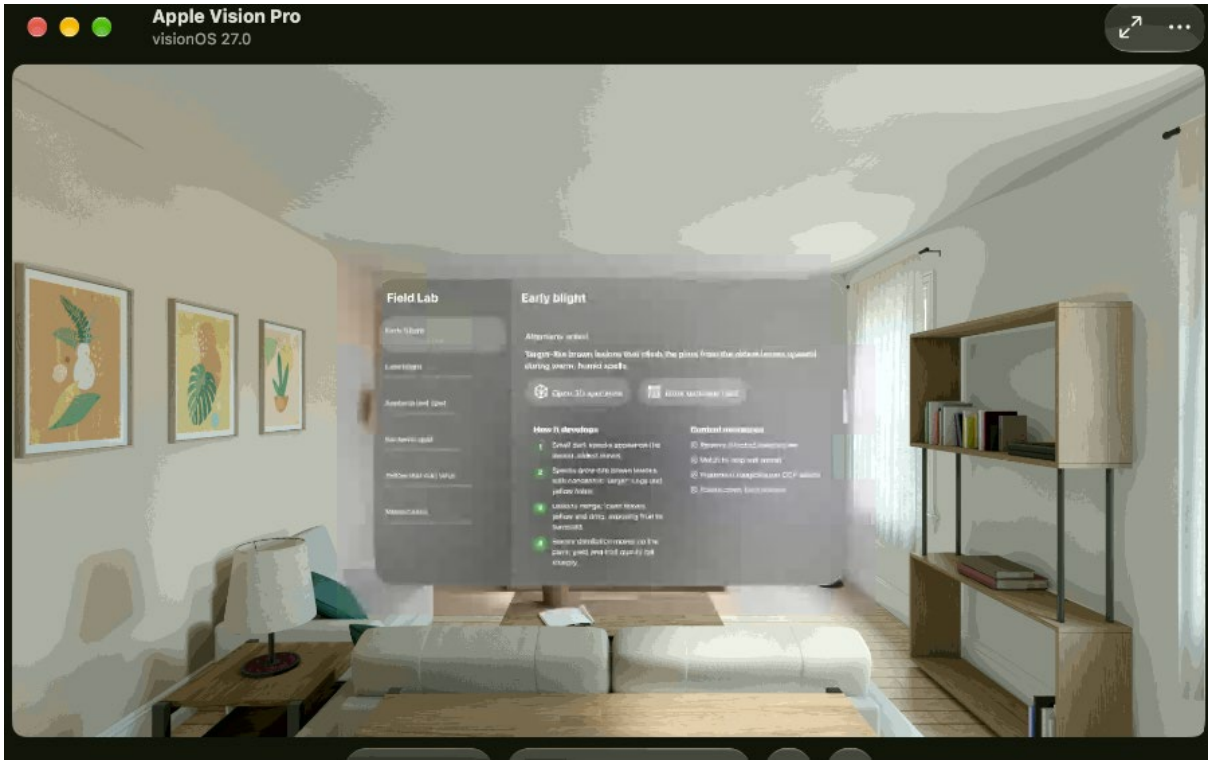


Figure 13: Field Lab lab window with the simulation controls and outbreak chart (Apple Vision Pro Simulator)



12. Existing similar applications

Table 8: Comparison with existing applications

Application	Platform	What it does	Gap Field Lab fills
Plantix (Peat → Helm AG)	Android	Photo-based detection of 500+ plant damage types on 45 crops [4]	Diagnosis only; no training, simulation or spatial content; no iOS/visionOS
PictureThis	iOS	Plant identification and disease diagnosis with care reminders; 4.8★ from ~1 M ratings; subscription [5]	Consumer houseplant focus; not for outbreak management or training
Govi Mithuru (Dialog)	Android, IVR 616	Sri Lankan advisory for 32+ crops; trilingual AI chat/voice and image pest identification (2025) [3]	Advisory content in 2D; no immersive training or what-if simulation
JigSpace	iOS, visionOS	3D presentations and interactive models for education and industry [7]	Generic 3D tool; no agronomic models, disease progression or epidemiology
Farm Everywhere (Nomtek concept)	Vision Pro concept	AR overlays of IoT farm data, 3D farm maps, VR safety training [6]	Concept for large equipment-heavy farms; not disease training for smallholder extension

Positioning. Field Lab is, to my knowledge, the first application combining (1) disease-stage visualisation on a manipulable 3D plant, (2) a weather-driven outbreak simulation at room scale and (3) intervention practice — localised to Sri Lankan crops and seasons and connected to an existing field-diagnosis app (KolaCare) for continuity from training to practice.

13. User experience and interaction design

visionOS interaction	Use in Field Lab	HIG principle followed
Look + pinch (indirect tap)	Inspect a leaf; spray a plant	Targets are whole leaves/plants with generous collision shapes; hover highlight via `HoverEffectComponent`
Pinch + drag	Rotate the specimen	Content moves with the hand; rotation is relative to where the drag began
Two-hand magnify	Zoom the specimen (0.6–1.6×)	Clamped scale keeps the plant within the volume
Ornaments	Stage slider and actions under the volume	Controls live outside the 3D content, on glass
Attachments	Leaf callout (billboarded) and floating scoreboard	SwiftUI text in space, always facing the user
Mixed immersion	Field on the real floor	User stays grounded; the room remains visible; content placed in front, nothing behind
Windows + volume + space	Each task in the scene type that suits it	Start in a window, progressively increase immersion; leaving is one button

Comfort: content is placed 1.1–4 m in front at floor level, nothing moves the user's viewpoint, animations are gentle (sway amplitude ≈ 3°), and the simulation advances in discrete days with speed control.

14. Technical implementation

14.1 Procedural 3D content

No 3D assets are used. `SpecimenController` builds a tomato plant from RealityKit primitives — pot, stem, five spiralling branch levels (golden-angle phyllotaxis) with three leaves each, and fruit trusses. For each leaf it pre-creates lesions (and halos) at seeded random positions; changing the disease stage only toggles entities and swaps cached materials, so scrubbing is instant. `SymptomModel` — a pure, unit-tested function — decides each leaf's colour, number of visible lesions, curl and size from the plant's progress and the leaf's age, so symptoms correctly start on old, lower leaves and climb the plant. Four symptom patterns cover the six diseases: spots with halos/rings, large blotches, mosaic mottling and leaf curl.

14.2 Entity-Component-System

Custom components (`LeafComponent`, `FieldPlantComponent`, `SwayComponent`, `RainDropComponent`) attach data to entities; two systems (`SwaySystem`, `RainSystem`) update all matching entities every rendered frame using `context.deltaTime`. Gestures use `targetedToAnyEntity()` and read components on the hit entity (walking up the hierarchy for field plants). The user's rotation is written into the sway component's base rotation, so the system and the gesture never fight over the transform.

14.3 Outbreak simulation

`OutbreakSimulation` is a deterministic compartment model over a 7 × 7 grid (healthy → incubating → infected → lost, with treated as an intervention), driven by a seeded SplitMix64 generator so every scenario can be replayed and unit tested.

```
weatherFactor  = tempSuitability(T) × moistureSuitability(RH, rain)      ∈ [0, 1]
pressure(i)    = Σ severity(j)  over infectious neighbours j within radius r
P(infect i)    = 1 − (1 − min(0.95, transmission × weatherFactor)) ^ pressure
severity(t+1)  = severity(t) + growthPerDay × max(0.25, weatherFactor)   → lost at 1.0
treated        : severity − 0.12/day, protected; only possible below 0.70 severity
```

Each disease has its own spread profile — for example late blight spreads two plants away (wind-borne spores) with 2-day incubation in cool, wet weather, while yellow leaf curl virus spreads via whiteflies in hot, dry weather. The reference model reproduces this behaviour (Figure below): under up-country mist late blight destroys the whole field in under three weeks, while in a Yala dry spell it barely spreads.

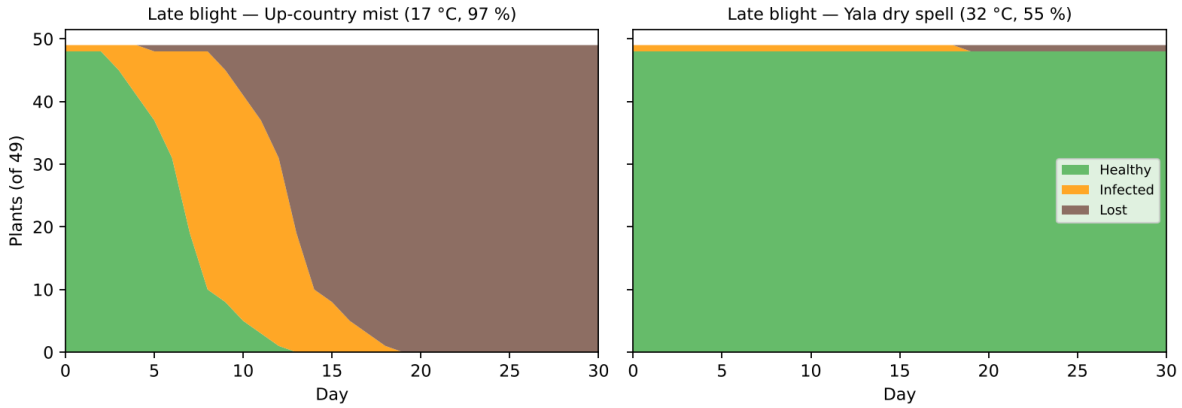


Figure 16: Outbreak curves for late blight under two scenarios (reference model, 49 plants, no treatment)

14.4 Performance and stability

- Meshes are created once and shared; `MaterialPalette` quantises colours so a few dozen material instances serve ~300 field entities.
- Field visuals update only when a per-plant state signature changes, even though the RealityView update closure can run often.
- The simulation runs in a cancellable main-actor task at 1–4 days per second and pauses automatically when the outbreak ends, at day 60, or when the immersive space closes.
- Scene lifecycle is tracked (`closed / inTransition / open`) so double-taps cannot open two immersive spaces.
- 19 unit tests cover the simulation (determinism, weather effects, treatment budget, roguing, neighbourhoods, invariants), the symptom model and the catalogue.

15. Pitch: business case

> **Value proposition**
>
> **Field Lab lets every agricultural trainer bring any crop disease — at any stage, in any season — into the classroom, and lets trainees practise outbreak decisions safely before real crops and incomes are at stake.**

15.1 Target market

Segment	Who	Why they buy
Primary — public extension (B2G)	Department of Agriculture in-service training institutes, provincial agriculture departments, farmer field schools	Standardised, repeatable training regardless of season; measurable trainee performance
Universities and colleges	Faculties of agriculture, agricultural technology courses	Practical plant-pathology labs without live pathogens
Agri-business (B2B)	Seed, fertiliser and crop-protection companies training field officers and dealers	Consistent, responsible-use training at scale
Development partners	NGOs and international programmes running farmer field schools in South and Southeast Asia	Reusable content for many crops and languages

The supporting signals are strong: agriculture employs about a quarter of Sri Lanka's workforce [2], disease losses are large and recurrent [1], and national advisory services are already investing in AI (Govi Mithuru's 2025 AI launch) [3]. Headset availability is the main constraint, so the go-to-market strategy targets institutions that buy a few shared devices rather than individual farmers.

15.2 Revenue model (proposed)

Stream	Model	Indicative pricing (assumption to validate)
Institutional licence	Annual subscription per training site; unlimited trainees; includes content updates	USD 1,200 per site per year
Content packs	Additional crops (tea, coconut, banana, cinnamon) and regional disease packs	USD 300 per pack per year
Training-as-a-service	Turn-key workshops with devices supplied, for NGOs and agri-businesses	Per-workshop fee
KolaCare (iOS) freemium	Free diagnosis; *Pro* adds multi-farm management, exports and instructor dashboards	LKR 450 per month

Content sponsorship by agri-input companies is possible only with strict editorial independence: treatment guidance must follow DOA recommendations and never promote brands.

15.3 Go-to-market strategy

1. **Pilot (months 0–6):** two pilots — a university faculty of agriculture and one DOA in-service training institute; measure knowledge gain with pre/post quizzes built into the app.

2. **Proof and content (months 6–12):** publish results; add rice, chilli and tea content; add SharePlay so a trainer and several trainees share one field.
3. **Scale in Sri Lanka (year 2):** provincial departments and agri-businesses; partner with an existing advisory channel (e.g. Govi Mithuru) for distribution; bundle with KolaCare iOS so trainees continue on their phones.
4. **Regional expansion (year 3):** South and Southeast Asian extension systems via development partners; iPad version of the field simulator to widen reach where headsets are scarce.

15.4 Competitive advantage and risks

Advantage	Risk	Mitigation
Unique combination of 3D pathology, epidemiological simulation and practice	Low headset penetration	Institution-owned shared devices; iPad port of the simulation; KolaCare iOS as the mass-market product
Procedural content — new diseases are data (JSON), not 3D modelling	Scientific accuracy of models	Advisory board of plant pathologists; parameters versioned and cited
Local crops, seasons and languages	Price sensitivity of public institutions	Grant/donor funding through development partners; tiered pricing

15.5 Impact

Better-trained extension officers diagnose earlier and advise more precise spraying — reducing crop loss, chemical cost and environmental harm. Because each officer serves many farmers, training a single officer multiplies the effect across a whole community.

15.6 Pitch deck outline (viva)

1. Title and hook — a field lost to late blight in one week
2. Problem — losses, scarce experts, seasonal diseases
3. Solution — Field Lab live demo
4. How it works — specimen, field, simulation
5. Market — segments and why now
6. Business model
7. Go-to-market and pilots
8. Competition and advantage
9. Roadmap — SharePlay, more crops, iPad
10. Team, ask and impact

16. Reflection

16.1 Design decisions I would defend

- **On-device first.** It costs model-size and accuracy trade-offs but guarantees privacy, zero running cost and offline use — the deciding factors for the target users.
- **Measure, don't just classify.** The colour analyser makes the app useful without a model and gives a severity figure that remains meaningful even when the classifier is unsure.
- **Data-driven content.** Diseases, risk profiles, symptom patterns and spread parameters live in JSON, so content can grow without code changes in both apps.
- **Procedural 3D.** Building plants from primitives avoided a 3D-modelling bottleneck and made every disease-stage combination possible.

16.2 Challenges

- Simulator limitations (no camera, no Neural Engine, no subject lifting) led to sample leaves, the colour fallback and compile-time guards — which also improved robustness.
- Field photos differ from training data; the quality gate and confidence warnings are honest about uncertainty rather than over-claiming.
- Coordinating RealityKit systems with gestures and SwiftUI state required clear ownership of each transform.
- Translating agronomic terms into Sinhala and Tamil needs expert review; names are included, longer descriptions remain English for now.

16.3 Future work

- Field-collected Sri Lankan images to fine-tune the model; on-device model updates with Background Assets.
- Widgets and Live Activities for risk alerts; App Intents/Siri for hands-free scanning.
- SharePlay multi-user field sessions and assessment mode in Field Lab.
- Instructor dashboard combining anonymised KolaCare reports into regional outbreak maps (opt-in).

17. AI-assisted development statement

As permitted by the assignment, I used an AI assistant (Claude by Anthropic) during development. The complete log, including prompts, is kept in `docs/AI_USAGE.md` in each repository. In summary:

Prompt (summary)	How the response was used	My verification
"Create the final application and final report; read the marking rubric carefully" with the brief attached; then chose Part A + B, plant-disease concept and Word report	Generated concept, architecture, SwiftUI/SwiftData/Vision/CoreML/RealityKit source, tests, disease catalogue, translations, icons, training scripts and the report draft	Built and ran both projects in Xcode, fixed compiler issues, ran all tests, captured real screenshots and measurements, trained the model myself, reviewed agronomic content and translations
☐ Add each further session	☐	☐

Responsibility for the submitted work, its correctness and my ability to explain every part of it in the viva remains mine.

References

[1] Down To Earth, "At least 40% global crops lost to pests every year: FAO", 3 June 2021 (reporting FAO, *Scientific review of the impact of climate change on plant pests*). https://www.downtoearth.org.in/news/agriculture/at-least-40-global-crops-lost-to-pests-every-year-fao-77252

[2] Trading Economics / World Bank, "Sri Lanka — Employment in agriculture (% of total employment)", 26.26 % (2023). https://tradingeconomics.com/sri-lanka/employment-in-agriculture-percent-of-total-employment-wb-data.html

[3] Dialog Axiata, "Dialog AI powers Govi Mithuru — Sri Lanka's first AI-powered agriculture advisory platform", 25 Sep 2025. https://www.dialog.lk/news/dialog-ai-powers-govi-mithuru-sri-lankas-first-ai-powered-agriculture-advisory-platform

[4] Future Farming, "Plantix: more than a crop doctor driven by AI". https://futurefarming.com/smart-farming/plantix-more-than-a-crop-doctor-driven-by-ai

[5] Apple App Store, "PictureThis – Plant Identifier". https://apps.apple.com/app/1252497129

[6] Nomtek, "Farm Everywhere — remote precision farming on Apple Vision Pro". https://www.nomtek.com/services/ar-vr-development/apple-vision-pro-development/use-cases/remote-precision-farming

[7] JigSpace, "JigSpace for Apple Vision Pro" and education use cases. https://www.jig.com/apple-vision-pro

[8] Department of Agriculture, Sri Lanka, "NAICC Services – 1920 Call Center". https://doa.gov.lk/naicc-services-1920-call-center

[9] Open-Meteo, Free Weather API (CC BY 4.0). https://open-meteo.com/

[10] D. P. Hughes and M. Salathé, "An open access repository of images on plant health to enable the development of mobile disease diagnostics", arXiv:1511.08060, 2015 (PlantVillage dataset: https://github.com/spMohanty/PlantVillage-Dataset).

[11] S. P. Mohanty, D. P. Hughes and M. Salathé, "Using deep learning for image-based plant disease detection", *Frontiers in Plant Science*, 7:1419, 2016.

[12] Apple Developer Documentation: SwiftData, Vision (VNGenerateForegroundInstanceMaskRequest, VNCoreMLRequest), Create ML, MapKit for SwiftUI, Swift Charts, TipKit, RealityKit and the Human Interface Guidelines for visionOS. https://developer.apple.com/documentation

[13] L. P. Smith, "Potato blight forecasting by 90 per cent humidity criteria", *Plant Pathology*, 5(3), 1956.

Appendix A — Rubric self-assessment

Criterion (marks)	Evidence	Section
App functionality (15)	Complete scan → diagnose → save → track → treat loop; weather alerts; map; library; settings; demo data; graceful fallbacks	1, 4, 7
Advanced UI & UX (15)	Custom animations, transitions, gestures, adaptive layouts, haptics, themes, accessibility	4, 5
Code quality & architecture (15)	MVVM + DI services, protocols, router, repository, versioned schema, logging, documented code, no dependencies	6, 7.6
Emerging technologies (15)	Vision subject lifting + CoreML classifier + pixel analysis; MapKit; Charts; TipKit; localisation	7.3–7.5
Testing & optimisation (10)	57 unit + 9 UI tests, performance metrics, Instruments, optimisation table, UAT plan	8
Documentation & reporting (10)	This report, README, user guide, model training guide, AI log	All
Innovation & creativity (10)	Objective severity and lesion map, weather-driven forecasting, trilingual, offline-first, spread zones	1, 7
Presentation & deployment (10)	Deployment checklist, privacy manifest, demo script	10
Part B: concept (25) / prototype (25) / UX (15) / technical (15) / pitch (20)	Sections 11–15	11–15

Appendix B — Repositories and build instructions

```
KolaCare/                  Part A — iOS 17+, Xcode 16+
├── KolaCare.xcodeproj     shared scheme, tests, coverage
├── KolaCare/              App · Models · Services · ViewModels
│                          DesignSystem · Views · Resources
├── KolaCareTests/         57 unit / integration / perf tests
├── KolaCareUITests/       9 UI tests
├── ModelTraining/         Create ML script, dataset prep
├── docs/                  user guide, AI usage log
└── project.yml, README.md

KolaFieldLab/              Part B — visionOS 2+
├── KolaFieldLab.xcodeproj
├── KolaFieldLab/          App · Models · Simulation
│                          RealityKit · Views · Resources
├── KolaFieldLabTests/     19 tests
└── README.md
```

1. Open the `.xcodeproj` in Xcode 16 or later.
2. Choose the scheme and a simulator (iPhone 16 for KolaCare; Apple Vision Pro for Field Lab).
3. Press ⌘R to run and ⌘U to test. For devices, set a development team under Signing & Capabilities.
4. Optional: train and add `LeafDoctor.mlmodel` (KolaCare `ModelTraining/README.md`).